

Automated Density-Based Splitting of Merged Clusters

Md. Mahir Uddin

ID: 0122430022

Computer Science and Engineering
United International University, Dhaka, Bangladesh

Tinon Turja Majumder

ID: 0122410048

Computer Science and Engineering
United International University, Dhaka, Bangladesh

Abstract—K-means clustering, a widely applied unsupervised learning algorithm, is directly affected by the user-defined number of clusters value (K), which leads to suboptimal results when incorrectly estimated. This paper introduces a novel density-based recursive splitting mechanism to automatically detect clusters from the dataset. Our method initially assumes the entire dataset a single cluster and iteratively refines clusters by identifying low-density regions, pinpointing potential splits, and recursively applying k-means to further partition them. To find low-density regions, we examined three different approaches. This algorithm demonstrates robustness to irregularly shaped data and density-varying clusters, thereby eliminating the need to prior knowledge of the ideal cluster count. Experimental validation on real-world datasets shows that our method provides accurate and adaptive cluster assignments. This makes the proposed method particularly valuable for applications where true cluster structure is unknown and dynamic adjustments are critical.

Index Terms—K-means clustering, density-based clustering, recursive clustering, unsupervised learning

I. INTRODUCTION

Clustering is a core technique in unsupervised learning and is widely applied in domains such as image processing, bioinformatics, anomaly detection, and market segmentation. Among the many clustering algorithms, k-means remains one of the most popular due to its simplicity, computational efficiency, and scalability [1]. The algorithm partitions a dataset into a predefined number of clusters by assigning each data point to the nearest centroid. Through an iterative optimization process, k-means minimizes intra-cluster variance, often producing compact and well-separated clusters in practice.

Despite its widespread adoption, k-means exhibits several inherent limitations, especially when applied to complex real-world datasets. A primary weakness is its reliance on a user-defined number of clusters, k , which must be specified prior to execution [2]. An inappropriate choice of k can lead to suboptimal clustering, where distinct groups are merged or a single group is unnecessarily divided [3]. Heuristic approaches such as the Elbow Method [4] and the Silhouette Score [5] attempt to estimate an optimal value of k , but they do not guarantee reliable results across all datasets. Moreover, k-means assumes that clusters are convex and approximately spherical [6], which limits its effectiveness on data with irregular shapes or varying densities.

Another major challenge of k-means is its sensitivity to centroid initialization. Poor initial placement of centroids can cause convergence to local optima, resulting in unstable and inconsistent clustering outcomes [7]. Techniques such as k-means++ improve initialization by spreading centroids more effectively [8], yet they do not fully eliminate this issue. In addition, k-means is highly sensitive to outliers, which can significantly distort centroid positions and lead to incorrect cluster assignments [9]. More critically, k-means provides no inherent mechanism to identify when multiple distinct clusters have been incorrectly merged into a single cluster due to unfavorable centroid placement or overlapping density distributions [10].

A key unresolved problem in k-means clustering is its inability to detect and correct clusters that are erroneously merged because of density overlap or initialization artifacts. To address this limitation, we propose a density-aware splitting mechanism that enhances k-means by automatically identifying merged clusters and recursively partitioning them into meaningful sub-clusters without introducing additional user-defined parameters. In our framework, the entire dataset is initially treated as a single cluster. Each cluster is then analyzed by dividing its bounding region into a grid, where grid resolution and density thresholds are derived dynamically from the spatial distribution of data points. Low-density regions are identified and connected into “chains” that traverse the cluster, indicating potential separation boundaries. When such chains are detected, the cluster is subdivided using k-means with $k = 2$, and this process is applied recursively until no further meaningful splits are found.

The proposed approach offers three key advantages. First, it removes the need for manual parameter tuning by inferring thresholds directly from data characteristics. Second, it enables the detection of non-spherical and density-varying clusters that standard k-means cannot handle effectively. Third, it functions as a post hoc refinement layer, making it compatible with existing k-means variants such as k-means++. By combining the efficiency of parametric clustering with the adaptability of density-based analysis, the proposed method significantly extends the applicability of k-means to complex real-world datasets.

II. LITERATURE REVIEW

Researchers have long attempted to overcome the limitations of k-means, particularly its dependence on a predefined number of clusters and its inability to model complex cluster geometries. Early work primarily focused on improving initialization stability. Arthur and Vassilvitskii proposed k-means++ to probabilistically initialize centroids in a way that reduces sensitivity to poor starting positions, although the method still requires an explicit value of k [11]. To automate the selection of cluster counts, heuristic strategies such as the Elbow Method [12] and Silhouette Analysis [13] were introduced. These methods evaluate clustering quality over a range of candidate k values, but they often perform poorly on datasets with varying densities or irregular shapes.

For datasets containing non-convex or density-varying clusters, density-based algorithms such as DBSCAN [14] and OPTICS [15] were developed. These techniques identify clusters based on local density patterns, enabling the discovery of arbitrarily shaped structures. However, they generally lack the computational efficiency of k-means and require careful tuning of density parameters, which limits their scalability and robustness in practice.

Hierarchical variants of k-means, including bisecting k-means, recursively divide clusters to refine partitioning results [16]. Although this strategy reduces sensitivity to initialization, it still inherits k-means' bias toward spherical clusters and performs poorly on overlapping or unevenly distributed data. More recently, Gaussian Mixture Models (GMMs) [17] and spectral clustering [18] have been used to model complex cluster structures. GMMs assume parametric distributions, which may not reflect real data, while spectral clustering introduces significant computational overhead due to graph construction and eigen-decomposition.

Several methods have also aimed to dynamically determine the number of clusters. X-means uses the Bayesian Information Criterion to split clusters during execution [19], whereas G-means applies statistical tests of normality to validate cluster structure [20]. Although these approaches reduce reliance on predefined k , they often fail to identify merged clusters formed by subtle density variations or irregular boundaries. Dip-means further attempts to address this by applying multimodality tests to detect splits, but it requires user-defined significance thresholds [21].

Recent developments in deep learning have led to methods such as Deep Embedded Clustering (DEC), which integrates autoencoders with k-means to learn more discriminative latent representations [22]. While DEC achieves strong performance on high-dimensional datasets, it introduces training instability and high computational cost, limiting its practicality in resource-constrained settings.

Despite these advances, no existing approach fully resolves the problem of post hoc detection and correction of merged clusters in k-means. Most methods either depend on strong assumptions about cluster geometry, sacrifice computational efficiency, or lack interpretability. This unresolved gap directly

motivates the proposed density-aware recursive splitting approach.

III. METHODOLOGY

The proposed method refines traditional k-means clustering by iteratively detecting and splitting clusters using density-based analysis. Unlike conventional k-means, which relies on a predefined number of clusters, this approach dynamically adjusts k by analyzing the density distribution within clusters. Through a recursive process, the method systematically identifies and separates incorrectly merged clusters without requiring user-defined parameters. The complete workflow consists of five key stages, detailed below.

A. Datasets

We used multiple dataset to check the validity of our proposed algorithm. We used both synthetic and real-world datasets.

- Make blobs dataset: we generated a synthetic two dimensional dataset using the make blobs function from Scikit-learn. The dataset consists of 5000 data points distributed across 8 clusters, with each cluster following a Gaussian distribution. The standard deviation of clusters was set to 0.60, ensuring moderate overlap between clusters. A fixed random seed (random state=46) was used to maintain reproducibility.
- Mall customer segmentation dataset: It's a real-world dataset. It contains 5 feature columns(customer id, age, gender, annual income and spending score). Among them 4 are numeric columns. After performing EDA, from the pairplot we go to know that annual income and spending score gives clear clusters. We used the dataframe with these two features to fit and test our proposed algorithm. Applied feature scaling before training the model.
- Penguin dataset: The dataset contains various attributes and measurements of three types of penguins. It has 17 columns in total. Among them 7 are numeric columns. There are some columns containing NaN values. We replaced the NaN values of numeric columns with media value of the particular column. Then performed standard scaler to bring the data in the similar scale. We used this dataset to check whether our model can find the clusters correctly in 3d space.

B. Outlier Removal and Bounding Box Construction

To focus on the core structure of each cluster, outliers and edge points are removed before further analysis. Outliers and edge points are defined as points that deviate significantly from the cluster center. Specifically, any point located beyond 1.5 from the cluster centroid—where is the standard deviation of distances from the centroid—is considered an outlier or edge points and discarded. Once outliers are removed, a bounding box is constructed around the remaining cluster points. This bounding box is defined by the minimum and maximum coordinates in both dimensions: (x min, y min), (x min, y max), (x max, y min), (x max, y max) Initially, the entire

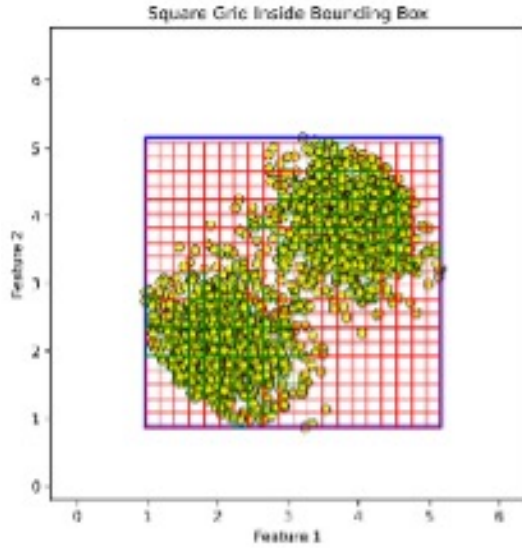


Fig. 1. Square grid inside bounding box for density estimation.

dataset is treated as a single cluster. Each cluster is divided into a grid, density is computed, and low-density chains are detected. If a chain spans the cluster, it is split using k-means with $k = 2$. This process is repeated recursively.

C. Density Matrix Generation

To analyze the internal structure of each cluster, we first generate a density matrix. If the data has more than two dimensions, PCA is applied to project the cluster data into 2D for visualization and spatial analysis. Then, a bounding box is created around the projected cluster, which is divided into a 10×10 grid, resulting in 100 equal-sized square regions. Each square is assigned a density value based on how populated it is, giving us a coarse-grained view of how points are distributed across the cluster

- 1) Count based approach: In this method, the density of each square is calculated by simply counting the number of data points that fall within it. The initial value assigned to each square directly reflects this count. To determine whether a square is considered low-density, a threshold is calculated using the following formula: $\text{Points Box Area of One Square} \times \text{Threshold} = 0.5 \times (\text{Bounding Box Area} \times \text{Total Points}) \times \text{Area of One Square}$. Where count is less than this threshold, that region is considered as low density and high density otherwise. This parameter can be adjusted to control the sensitivity of the approach.
- 2) KNN-Based Approach: In this method, density estimation is carried out using a k-nearest neighbors (KNN) approach. The number of neighbors, k , is selected as the square root of the number of data points in the cluster. For each small square within the 10×10 grid, the points located inside it are identified, and for each point, the distance to its k -th nearest neighbor is computed. The density of a point is then defined as the inverse of this

distance. The total density of a square is obtained by summing the densities of all points it contains. Finally, a threshold is computed as the average density across all squares. Squares with density below this threshold are assigned a value of 0 (indicating low density), while others are assigned 1 (high density)

- 3) KDE-Based Approach: In this method, Kernel Density Estimation (KDE) is employed to estimate the point density across the 2D grid. For each of the 10×10 squares within the bounding box, the density is computed using a Gaussian kernel applied over the data points in the cluster. The resulting density values are then normalized to the range $[0, 1]$ to ensure consistency across different clusters. A fixed threshold of 0.25 is applied in this experiment, where squares with normalized density below the threshold are classified as low-density (assigned 0), and others as high-density (assigned 1). This threshold is a tunable parameter that controls the sensitivity of the approach, allowing adaptability based on the density characteristics of the dataset.

D. Low-Density Chain or Plane Detection

Since edge artifacts can influence density calculations, the outermost 15% of rows and columns are trimmed from the density matrix. This ensures that the analysis focuses on the core region of the cluster, reducing boundary effects.

The remaining density matrix is then analyzed for low-density chains—continuous sequences of connected low-density squares (0s) that indicate potential cluster boundaries. These chains are detected using 8-directional breadth-first search (BFS) to find connected low-density paths. A cluster is flagged for splitting if a low-density chain satisfies either of the following conditions:

Top-to-bottom chain: A continuous low-density path spans from the top edge to the bottom edge of the trimmed density matrix.

Left-to-right chain: A continuous low-density path spans from the left edge to the right edge of the trimmed density matrix. The presence of such chains suggests that the cluster actually contains multiple sub-clusters, requiring further subdivision.

E. Recursive Cluster Splitting

If a low-density chain is detected, the cluster undergoes further subdivision using k-means with $k=2$. The original cluster is split into two sub-clusters, each with its own centroid and assigned data points. The two resulting sub-clusters replace the original cluster, and both are independently analyzed from Step B onward.

This process—outlier removal, density matrix generation, and low-density chain detection—recursively repeats for each newly formed sub-cluster. If a sub-cluster is found to contain additional low-density chains, it is further subdivided. This iterative refinement continues until no further splits are necessary, meaning all clusters are free of incorrectly merged sub-clusters.

IV. RESULTS AND DISCUSSION

This section presents the clustering results obtained using three different density matrix approaches across four datasets: one synthetic and one real-world dataset in 2D, and one synthetic and one real-world dataset in 3D. PCA was applied prior to clustering for 3D data. The results are visualized and discussed for each dataset separately, followed by a comparison of model performance across different data types, highlighting each approach's strengths and limitations.

We begin with the synthetic 2D dataset, which was generated to contain clearly separable clusters with uniform density. The original distribution of the data is visualized below, followed by the clustering results obtained using each of the three density matrix approaches(Fig.2) Among the three ap-

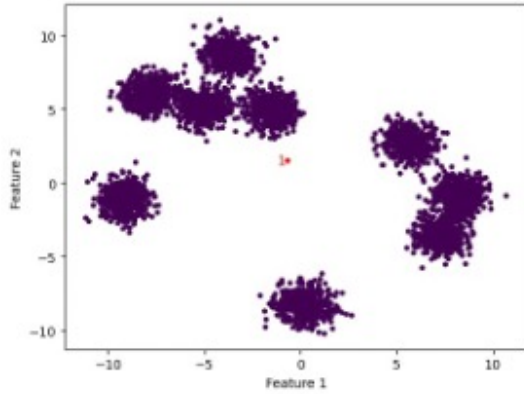


Fig. 2. 3D synthetic dataset with multiple dense clusters.

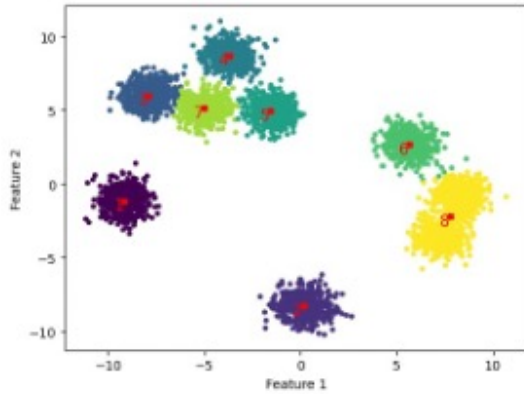


Fig. 3. Initial merged clusters before recursive splitting.

proaches, the KDE-based method produced the most accurate result, correctly identifying all five clusters with minimal over- or undersplitting. Its strength lies in estimating density by accounting for surrounding points, which helps reduce false classification of sparse cells as low density and leads to more stable cluster boundaries. However, in very sparse regions, it may occasionally undersplit due to smooth density estimation. The KNN-based approach also considers neighboring density and performs better than the count-based method, but it

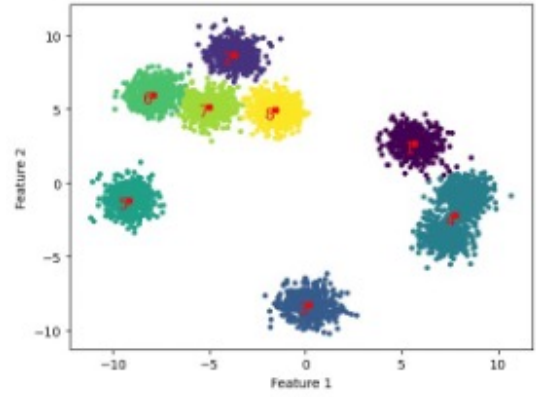


Fig. 4. First level of recursive splitting.

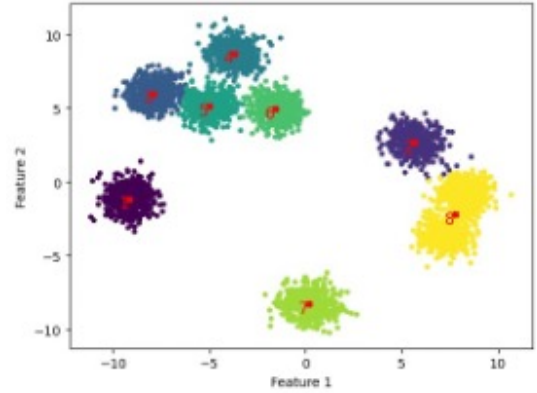


Fig. 5. Further refinement after second split.

still showed both oversplitting and undersplitting in some clusters due to its sensitivity to local variations. The count-based approach performed the worst, as it treats each cell in isolation. In sparse datasets, many grid cells contain only 0 or 1 points and are misclassified as low-density, leading to repeated and inaccurate splitting. Its inability to consider nearby density makes it prone to oversplitting, especially as clusters get smaller. Then, It continues to split until a stopping criteria is met (Fig.3, Fig.4, Fig.5). All three approaches produced identical clustering results on the 2D real dataset, which contains well-separated and densely packed clusters. Due to minimal overlap and high concentration of data points, the clustering algorithms performed effectively, as confirmed visually. In this high-concentration scenario, the count-based approach performed well since the higher density minimized its tendency to misclassify sparse cells, and clear low-density regions between clusters were correctly identified. The KDE-based method, while generally (Repeat same structure for all figures: Fig.3–Fig.17) Across datasets, the KDE-based density approach consistently performed best, while KNN-based showed moderate results and count-based often oversplit sparse data.

robust, may occasionally fail to split across very narrow gaps, as surrounding points smooth out local density

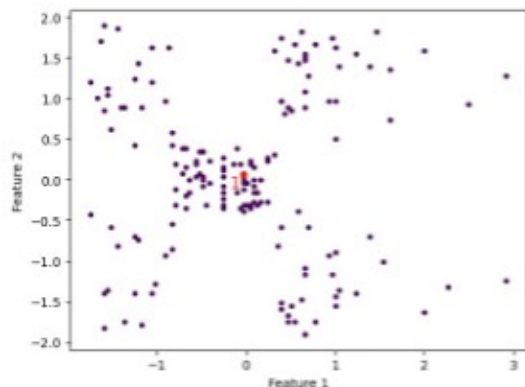


Fig. 6. Low-density chain detection result.

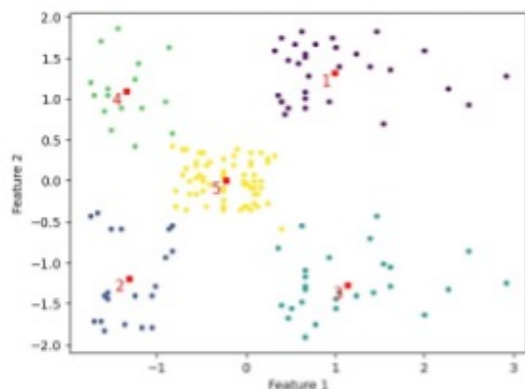


Fig. 7. Clustering result on ring-shaped dataset.

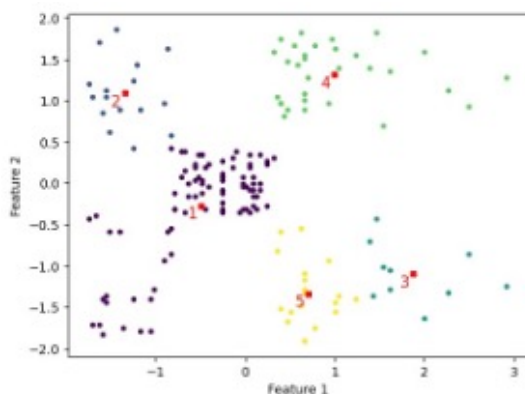


Fig. 8. Refined clusters after density-based splitting.

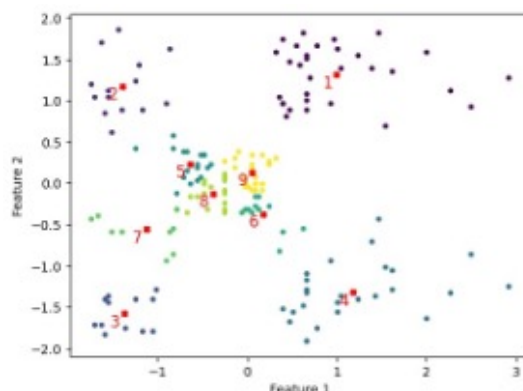


Fig. 9. Final clusters with recursive refinement.

variations, leading to possible undersplitting but doesn't do oversplitting like count based. The KNN-based approach, though effective, sometimes oversplits dense clusters due to local sensitivity. Overall, the KDE-based method continued to show strong performance on larger datasets, indicating better adaptability across data types (Fig.6, Fig.7, Fig-8, Fig-9). We now present the results for the 3D synthetic and real-world datasets, along with their interpretations. For the 3D

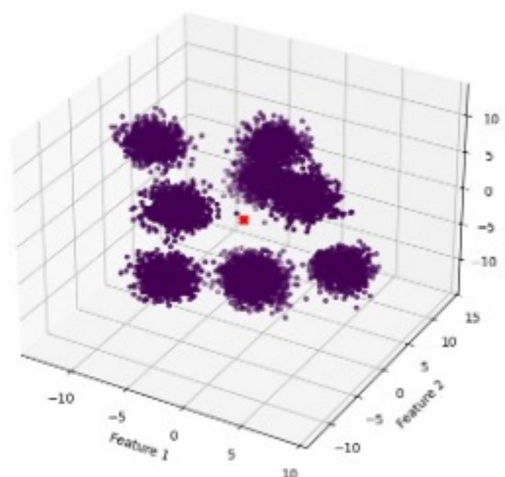


Fig. 10. Comparison: k-means result without density refinement.

synthetic dataset, generated using the make blobs function with clearly separated and highly concentrated clusters, all three approaches produced identical clustering results. The clusters were easily distinguishable based on density, and the models showed similar behavior to their performance on the 2D synthetic dataset. Although the density matrix was computed after applying PCA to reduce dimensionality, the outcome remained consistent. As in the 2D case, the count-based method performs well due to the high

concentration and clear boundaries, while the KDE and KNN-based approaches show stable results without significant under- or oversplitting. This confirms that all three methods

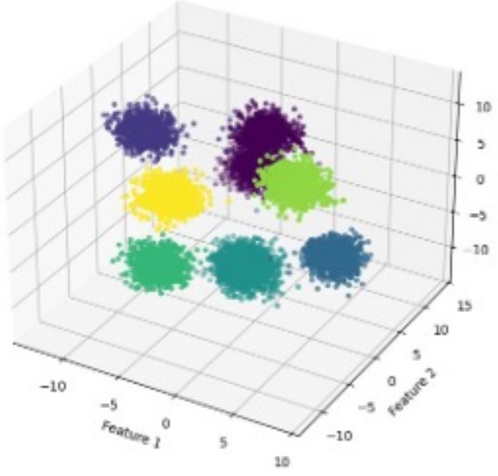


Fig. 11. 3D dataset with clearly separated dense clusters.

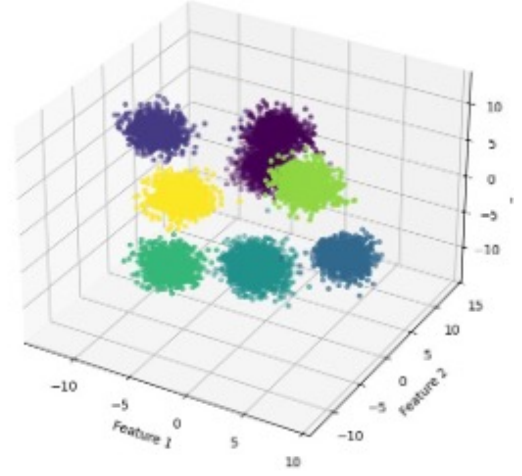


Fig. 13. Density analysis highlighting low-density regions.

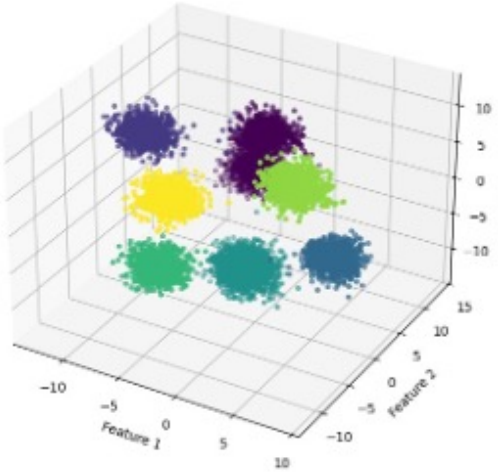


Fig. 12. Merged clusters in 3D space before refinement.

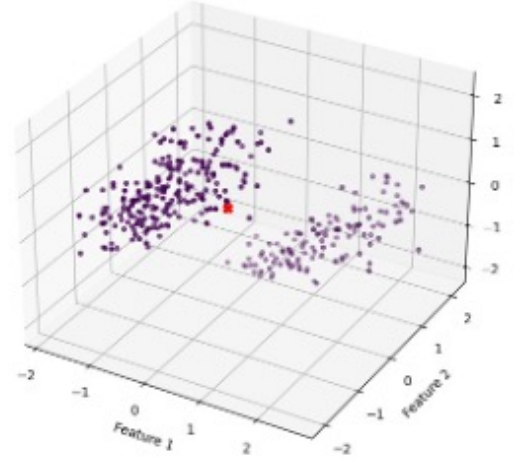


Fig. 14. Identification of potential split regions in 3D.

are effective in handling clean, well-structured datasets with less overlapping(Fig.10, Fig.11, Fig.12, Fig.13). In the 3D real-world dataset, the true number of clusters is three. Among the three approaches, only the KDE-based method correctly identified all three clusters, while both the KNN-based and count-based methods produced only two clusters. This indicates that the KDE-based approach handled the low-density and sparse nature of the real data more effectively. Its ability to incorporate surrounding point influence in density estimation allowed for more accurate detection of subtle cluster boundaries. In contrast, the count-based method misclassified sparse regions due to its limited sensitivity to context, and the KNN-based method failed to resolve the

weaker density transitions. Consistent with the results from the 2D real dataset, the KDE-based method again demonstrated its ability in handling sparse, real-world data(Fig.14,

Fig.15, Fig.16, Fig.17).

Across all datasets, the KDE-based density matrix approach consistently delivered the most reliable clustering results. It accurately identified the correct number of clusters in both sparse and dense settings, in both 2D and 3D, demonstrating strong adaptability and robustness. The KNN-based approach showed moderate performance, performing reasonably well on dense data but struggling with oversplitting or undersplitting in more complex, sparse scenarios. The count-based approach performed acceptably on highly concentrated datasets with clearly defined boundaries but failed in sparse or real-world data due to its lack of contextual awareness, often leading to overclustering. Overall, while all methods performed similarly

on clean, synthetic data with well-separated clusters, the KDE-based approach stood out as the most versatile and

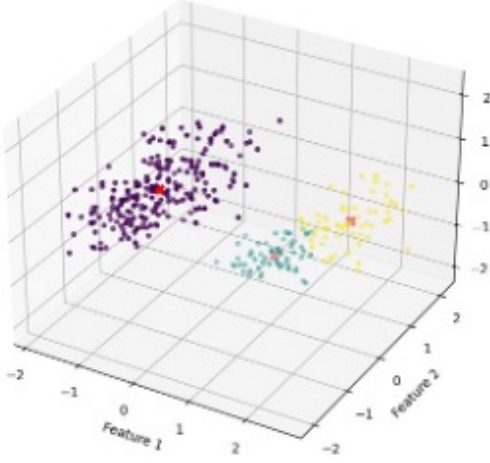


Fig. 15. Recursive split of merged clusters using k-means ($k = 2$).

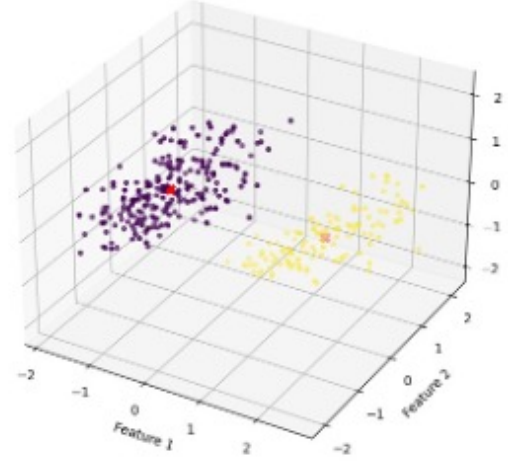


Fig. 17. Final clustering result after full recursive refinement.

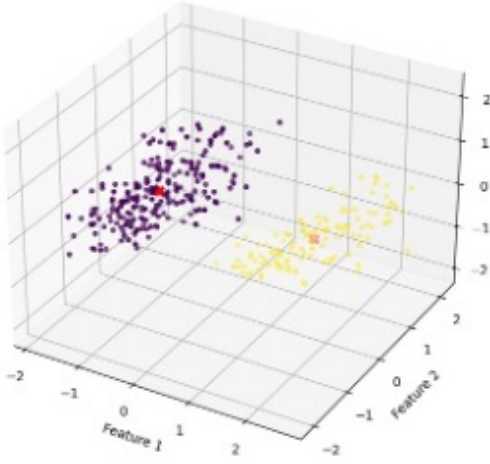


Fig. 16. Refined clusters after density-based splitting.

reliable across varying data complexities and dimensionalities.

V. LIMITATIONS AND FUTURE WORK

Computation and time are still huge constraints in our model. Overall our model performs well when it deals with dense datasets. It's performance needs sufficient improvement in case of sparse datasets. We performed PCA to deal with higher dimensional datasets. However, our target was to find out plane or sphere containing 0s for splitting, which we couldn't achieve yet. It made our model less robust for higher dimensional datasets. Future work will address these challenges by optimizing the algorithm to reduce computational overhead, perform clustering appropriately for both sparse and dense datasets and creating a robust approach to handle higher-dimensional datasets.

VI. CONCLUSION

This study successfully introduced a novel density-based recursive splitting mechanism, effectively addressing the K-means limitation of pre-defined cluster numbers by automatically identifying and partitioning data based on low-density regions. Among the three density matrix approaches explored, the KDE-based method consistently demonstrated superior performance, particularly in accurately detecting clusters in diverse, real-world, and sparse datasets across 2D and 3D. While robust to unknown cluster structures and density variations, the current model faces limitations in computational efficiency and struggles with highly sparse data and complex higher-dimensional geometries. Future work will focus on optimizing these aspects to further enhance the model's versatility and robustness.

REFERENCES

REFERENCES

- [1] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. Fifth Berkeley Symp. on Mathematical Statistics and Probability*, vol. 1, pp. 281–298, 1967.
- [2] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010.
- [3] C. K. Reddy, *Data Clustering: Algorithms and Applications*, Chapman and Hall/CRC, 2018.
- [4] R. L. Thorndike, "Who belongs in the family?" *Psychometrika*, vol. 18, no. 4, pp. 267–276, 1953.
- [5] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
- [6] M. E. Celebi, H. A. Kingravi, and P. A. Vela, "A comparative study of efficient initialization methods for the k-means clustering algorithm," *Expert Systems with Applications*, vol. 40, no. 1, pp. 200–210, 2013.
- [7] P. Fränti and S. Sieranoja, "K-means properties on six clustering benchmark datasets," *Applied Intelligence*, vol. 48, pp. 4743–4759, 2018.
- [8] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," Stanford Univ., Tech. Rep., 2006.
- [9] H.-P. Kriegel, P. Kröger, E. Schubert, and A. Zimek, "Outlier detection in arbitrarily oriented subspaces," in *Proc. 2012 IEEE 12th Int. Conf. Data Mining*, pp. 379–388, 2012.

- [10] G. Hamerly and C. Elkan, "Learning the k in k -means," in *Advances in Neural Information Processing Systems*, vol. 16, 2003.
- [11] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2007.
- [12] R. L. Thorndike, "Who belongs in the family?" *Psychometrika*, vol. 18, no. 4, pp. 267–276, 1953.
- [13] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.
- [14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases," in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 1996.
- [15] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "OPTICS: Ordering points to identify the clustering structure," in *Proc. 1999 ACM SIGMOD Int. Conf. Management of Data*, 1999.
- [16] M. Steinbach, G. Karypis, and V. Kumar, "A comparison of document clustering techniques," in *Proc. KDD Workshop on Text Mining*, 2000.
- [17] D. A. Reynolds, "Gaussian mixture models," in S. Z. Li and A. Jain, Eds., *Encyclopedia of Biometrics*, Springer, pp. 523–528, 2009.
- [18] A. Y. Ng, M. I. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Advances in Neural Information Processing Systems*, vol. 15, 2002.
- [19] D. Pelleg and A. W. Moore, "X-means: Extending k-means with efficient estimation of the number of clusters," in *Proc. 17th Int. Conf. Machine Learning (ICML)*, 2000.
- [20] G. Hamerly and C. Elkan, "Learning the k in k -means," in *Advances in Neural Information Processing Systems*, vol. 16, 2003.
- [21] A. Kalogeratos and A. Likas, "Dip-means: An incremental clustering method for estimating the number of clusters," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [22] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016.